

Coin Toss

Finance and Budgeting Manager

Preliminary Project Specification

A. System Description

1. What the System Does

Coin Toss is a personal finance and budgeting web application. It helps users track their income, expenses, budgets, and savings goals in one place. The system provides a visual dashboard with charts, analytics, and notifications to give users a clear picture of their financial health. It replaces spreadsheets and complex budgeting software with a simple, intuitive interface accessible from both desktop and mobile devices.

2. Target Audience

People who want a clear picture of where their money goes. This includes young professionals starting to budget, freelancers tracking income from multiple sources, families managing household finances, and anyone who prefers a simple visual tool over complicated accounting software.

3. What Need It Addresses

Managing personal finances with spreadsheets is error-prone, hard to visualize, and difficult to maintain. Existing budgeting apps are often too complex or locked behind paywalls. Coin Toss solves this by providing a free, simple, visual tool that automatically tracks spending, sets budgets, monitors savings goals, and sends alerts when budgets are close to their limits.

B. Wireframes / Mockups

The design follows a Minimalist SaaS Dashboard style with a warm brown color scheme (#704c35). The app uses card-based layouts, chart visualizations, and a responsive design that adapts between desktop (sidebar navigation) and mobile (bottom navigation bar).

1. Dashboard View

The main landing page after login. Features a left sidebar (desktop) or bottom nav (mobile) with navigation links: Dashboard, Transactions, Budgets, Goals, Analytics, Notifications, Settings. Main content area displays four KPI cards at the top: Balance, Income, Expenses, and Savings. Below the KPI cards, two charts are displayed side by side: a bar chart showing this month income vs expenses, and a pie chart showing the income vs expense breakdown. A budget usage progress bar appears below the charts. At the bottom, a list of the five most recent transactions is shown.

Dashboard		Health: 72	
Balance	Income	Expense	Save
\$12,450	\$5,230	\$3,180	\$2,050

Income vs Expense (Bar Chart)	Income vs Expense (Pie/Donut)
Budget Usage: 48% of \$3,000 [=====]	
Recent Transactions	
Grocery Store	-\$85.00 Jun 15
Salary	+\$12,000 Jun 01
Electric Bill	-\$320.00 Jun 03

2. Transactions Page

Displays a searchable, filterable table (desktop) or card list (mobile) of all transactions. Features a search bar, type filter (All / Income / Expense / Transfer), and pagination (15 per page). Each row shows description, category, date, type badge, and amount. Edit and delete buttons appear on each row. An Add Transaction button opens a dialog form.

Transactions	[+ Add Transaction]		
[Search]	[Type: All v]		
Description	Category	Date	Type
Grocery Store	Food	Jun 15	EXP
Salary	Work	Jun 01	INC
Electric Bill	Utilities	Jun 03	EXP
< Page 1 of 3 >			

3. Budgets Page

Shows budget cards in a 2-column grid (desktop) or single column (mobile). Each card displays the category name with a color dot, the spent amount vs limit, and a progress bar. Progress bars change color: normal (brown) under 90%, warning (yellow) at 90-99%, and exceeded (red) at 100%+. An Add Budget button opens a dialog form.

Budgets	[+ Add Budget]		
Food	Rent		
\$450 / \$600	\$1,200 / \$1,200		
[===== 75%]	[===== 100%]		
[Edit] [Delete]	[Edit] [Delete]		

4. Goals Page

Shows savings goal cards in a 2-column grid (desktop) or single column (mobile). Each card shows the goal name, current amount vs target amount, a progress bar with percentage, an optional deadline, and buttons to Contribute, Edit, and Delete. The Contribute button opens a small dialog to add funds.

Savings Goals	[+ Add Goal]		
Vacation	Emergency		
\$1,200 / \$3,000	\$800 / \$5,000		
[===== 40%]	[== 16%]		
Due: Sep 15			
[Contribute]	[Contribute]		
[Edit] [Delete]	[Edit] [Delete]		

| +-----+ +-----+ |
+-----+

5. Analytics Page

Displays four KPI cards (average spending, savings rate, health score, net worth), followed by an area chart showing monthly spending trends over six months, a pie chart showing expense breakdown by category, and a bar chart showing cash flow over twelve months. All charts stack vertically on mobile.

6. Additional Pages

Login: Centered card on a gradient background with email, password, remember me checkbox, and links to register and forgot password.

Register: Similar layout with name, email, password, and confirm password fields.

Settings: Six sections - Profile, Password, Appearance (dark mode toggle), Currency and Locale, Session (sign out), and Danger Zone (delete account).

Notifications: List of notifications with unread count badge, mark all read button, and delete per notification.

C. DB Schema (ERD)

The database uses PostgreSQL (via Supabase) with 8 tables. All money amounts are stored in USD using Decimal(12,2). Passwords are hashed with bcrypt (cost factor 12).

1. users

```
{
  id:          UUID,          // Primary key, auto-generated
  name:        String,        // Display name (2-100 chars)
  email:       String,        // Login email, unique
  password_hash: String,      // Bcrypt hash (cost 12)
  role:        Enum,          // 'USER' | 'ADMIN' (default: 'USER')
  is_verified: Boolean,       // Email verified? (default: false)
  is_locked:   Boolean,       // Account locked? (default: false)
  failed_logins: Integer,     // Consecutive failed attempts (default: 0)
  lock_until:  Timestamp,     // Lock expiry time (null if not locked)
  reset_token: String,        // SHA-256 hash of reset token (null if none)
  reset_expires: Timestamp,   // Reset token expiry (1 hour)
  currency:    String,        // Display currency (default: 'USD')
  locale:      String,        // Locale for formatting (default: 'en-US')
  token_version: Integer,     // Incremented to invalidate tokens (default: 0)
  created_at:  Timestamp,     // Auto-generated
  updated_at:  Timestamp      // Auto-updated
}
```

2. transactions

```
{
  id:          UUID,          // Primary key
  user_id:     UUID,          // FK -> users.id (cascade delete)
  category_id: UUID,          // FK -> categories.id (set null on delete)
  amount:      Decimal(12,2), // Amount in USD, always positive
  description: String,        // 1-255 chars
  type:        Enum,          // 'INCOME' | 'EXPENSE' | 'TRANSFER'
  date:        Timestamp,     // When it happened
  payment_method: String,     // Optional (max 50 chars)
  notes:       String,        // Optional (max 1000 chars)
  receipt_url: String,        // Vercel Blob URL (optional)
  is_recurring: Boolean,      // Auto-created by cron? (default: false)
  tags:        String[],      // User tags (max 10, each max 30 chars)
  created_at:  Timestamp,
  updated_at:  Timestamp
}
Indexes: [user_id, date], [user_id, type], [user_id, category_id],
         [user_id, date, type]
```

3. categories

```
{
  id:          UUID,          // Primary key
  user_id:     UUID,          // FK -> users.id (null = system default)
  name:        String,        // 1-50 chars
  icon:        String,        // Lucide icon name (default: 'circle')
  color:       String,        // Hex color (default: '#6366f1')
  type:        Enum,          // 'INCOME' | 'EXPENSE'
  created_at:  Timestamp,
  updated_at:  Timestamp
}
Unique: [user_id, name, type] -- prevents duplicates per user
```

4. budgets

```
{
  id:          UUID,          // Primary key
  user_id:     UUID,          // FK -> users.id (cascade delete)
  category_id: UUID,          // FK -> categories.id (restrict delete)
  limit:       Decimal(12,2), // Max spending amount in USD
  period:      Enum,          // 'WEEKLY' | 'MONTHLY' | 'YEARLY'
  start_date:  Timestamp,     // Period start (default: now)
  end_date:    Timestamp,     // Period end (default: end of month)
  created_at:  Timestamp,
  updated_at:  Timestamp
}
Unique: [user_id, category_id, period]
Computed: spent = sum of expenses in category within date range
          percentage = (spent / limit) * 100
```

5. savings_goals

```
{
  id:          UUID,          // Primary key
  user_id:     UUID,          // FK -> users.id (cascade delete)
  name:        String,        // Goal name (1-100 chars)
  target_amount: Decimal(12,2), // Savings target in USD
  current_amount: Decimal(12,2), // Amount saved (default: 0, atomic update)
  deadline:    Timestamp,     // Optional target date (null = no deadline)
  created_at:  Timestamp,
  updated_at:  Timestamp
}
Computed: progress = (current_amount / target_amount) * 100
```

6. recurring_transactions

```
{
  id:          UUID,          // Primary key
  user_id:     UUID,          // FK -> users.id (cascade delete)
  category_id: UUID,          // FK -> categories.id (restrict delete)
  amount:      Decimal(12,2), // Amount per generated transaction
  description: String,        // Description for generated transactions
  type:        Enum,          // 'INCOME' | 'EXPENSE' | 'TRANSFER'
  interval:    Enum,          // 'DAILY' | 'WEEKLY' | 'MONTHLY' | 'YEARLY'
  day_of_month: Integer,      // For MONTHLY/YEARLY (1-31)
  day_of_week: Integer,       // For WEEKLY (0=Sun - 6=Sat)
  start_date:  Timestamp,     // When to start (default: now)
  end_date:    Timestamp,     // When to stop (null = forever)
  next_date:   Timestamp,     // Next due date
  is_active:   Boolean,       // Still running? (default: true)
  created_at:  Timestamp,
  updated_at:  Timestamp
}
Index: [next_date] -- cron job queries due templates
```

7. notifications

```
{
  id:          UUID,          // Primary key
  user_id:     UUID,          // FK -> users.id (cascade delete)
  title:       String,        // Short title
  message:     String,        // Full message body
  type:        Enum,          // 'INFO' | 'WARNING' | 'ERROR' | 'SUCCESS'
  is_read:     Boolean,       // Read by user? (default: false)
```

```

    created_at: Timestamp,
    updated_at: Timestamp
}
Indexes: [user_id], [is_read]

```

8. refresh_tokens

```

{
  id:          UUID,          // Primary key
  user_id:     UUID,          // FK -> users.id (cascade delete)
  token:       String,        // Refresh token string (unique)
  family:      String,        // Family ID (same login = same family)
  expires_at:  Timestamp,     // Token expiry
  is_revoked:  Boolean,        // Revoked? (default: false)
  created_at:  Timestamp,
  updated_at:  Timestamp
}
Indexes: [user_id], [token] (unique)
Rules: Reusing a revoked token revokes the entire family (theft detection)

```

Table Relationships Summary

From	To	Type	On Delete
User	Transaction	1 to many	Cascade
User	Category	1 to many	Cascade
User	Budget	1 to many	Cascade
User	SavingsGoal	1 to many	Cascade
User	RecurringTxn	1 to many	Cascade
User	Notification	1 to many	Cascade
User	RefreshToken	1 to many	Cascade
Category	Transaction	1 to many	Set Null
Category	Budget	1 to many	Restrict
Category	RecurringTxn	1 to many	Restrict

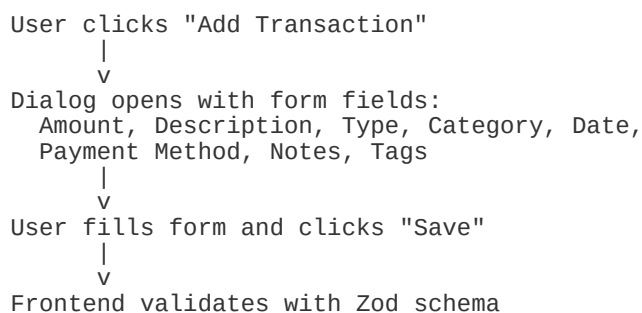
D. User Flow

1. Registration and Login Flow

```

User opens the app
|
v
Does the user have an access token (login session)?
+-- No --> Show /login page
|
|   +-- User enters email + password --> Correct?
|   |   +-- Yes --> Store tokens --> Go to /dashboard
|   |   +-- No --> Show error message
|   +-- "Create account" link --> /register
|       +-- Fill name + email + password
|       +-- Account created --> Go to /login
|
+-- "Forgot password?" --> /forgot-password

```



```

|
+-- Validation fails --> Show field errors
|
+-- Validation passes --> API POST /transactions
|
|               +-- 201 Created --> Close dialog,
|               |               refresh transaction list
|
+-- Error --> Show error message in dialog

```

5. Category Deletion Flow

User clicks "Delete" on a category

```

|
v
Is it a system default category?
|
+-- Yes --> Return 403 Forbidden (cannot delete system categories)
|
+-- No --> Start database transaction:
|           1. Find or create "Uncategorized" fallback of same type
|           2. Reassign all transactions to fallback
|           3. Reassign all budgets to fallback
|           4. Reassign all recurring templates to fallback
|           5. Delete the category
|           6. Commit (all steps succeed or all fail)

```

6. Recurring Transactions (Daily Cron Job)

Cron job fires at midnight (Vercel Cron)

```

|
v
Check X-Cron-Secret header matches server config?
|
+-- No --> Return 401 (reject)
|
+-- Yes --> Find all active recurring templates
|           where next_date <= now()
|           |
|           +-- None found --> Return { processed: 0 }
|           +-- Found N templates --> For each:
|               1. Create a new Transaction (copy fields)
|               2. Advance next_date to next interval
|               3. If next_date > end_date, set is_active = false
|               4. Return { processed: N }

```

E. API Endpoints

Base URL: /api/v1. Auth required for most endpoints (Bearer token in Authorization header).
All money amounts in USD.

Method	Endpoint	Description	Body / Query
POST	/auth/register	Create a new account	name, email, password, passwordConfirmation
POST	/auth/login	Log in and get tokens	email, password, rememberMe?
POST	/auth/logout	Log out, revoke tokens	refreshToken? (or cookie)
POST	/auth/refresh	Get new access token	refreshToken? (or cookie)
POST	/auth/forgot-password	Request password reset email	email
POST	/auth/reset-password	Reset password with token	token, password, passwordConfirmation
GET	/auth/me	Get logged-in user profile	-

(continued)

Method	Endpoint	Description	Body / Query
PATCH	/auth/profile	Update name and/or email	name?, email?
PATCH	/auth/me/password	Change password	currentPassword, newPassword, newPasswordConfirmation
PATCH	/auth/preferences	Update currency/locale	currency?, locale?
DELETE	/auth/account	Delete account permanently	-
GET	/transactions	List transactions (paginated)	page, limit, sortBy, type, search, ...
GET	/transactions/summary	Get totals for date range	startDate?, endDate?
GET	/transactions/:id	Get one transaction	-
POST	/transactions	Create a transaction	amount, description, type, categoryId?, ...
PUT	/transactions/:id	Update a transaction	amount?, description?, ...
DELETE	/transactions/:id	Delete a transaction	-
POST	/transactions/bulk-delete	Delete multiple	ids[]
GET	/categories	List all categories	-
GET	/categories/:id	Get one category	-
POST	/categories	Create a category	name, type, icon?, color?
PUT	/categories/:id	Update a category	name?, icon?, color?
DELETE	/categories/:id	Delete a category (reassigns)	-
GET	/budgets	List budgets (paginated)	page, limit
GET	/budgets/:id	Get one budget	-
POST	/budgets	Create a budget	categoryId, limit, period?, ...
PUT	/budgets/:id	Update a budget	limit?, period?, ...
DELETE	/budgets/:id	Delete a budget	-
GET	/goals	List goals (paginated)	page, limit
GET	/goals/:id	Get one goal	-
POST	/goals	Create a goal	name, targetAmount, currentAmount?, deadline?
PUT	/goals/:id	Update a goal	name?, targetAmount?, ...
DELETE	/goals/:id	Delete a goal	-

(continued)

Method	Endpoint	Description	Body / Query
POST	/goals/:id/contribute	Add money to a goal	amount
GET	/notifications	List notifications (paginated)	page, limit
PATCH	/notifications/read-all	Mark all as read	-
PATCH	/notifications/:id/read	Mark one as read	-
DELETE	/notifications/:id	Delete a notification	-
GET	/analytics/dashboard	Dashboard summary + KPIs	-
GET	/analytics/overview	Combined dashboard + monthly	-
GET	/analytics/monthly-spending	Income/expense by month	months? (default: 6)
GET	/analytics/category-breakdown	Expense breakdown by category	startDate?, endDate?
GET	/analytics/cash-flow	Income vs expenses over time	months? (default: 12)
GET	/analytics/net-worth	Net worth + monthly trend	-
POST	/uploads/receipt	Upload receipt file (multipart)	file (multipart/form-data)
POST	/uploads/receipt/delete	Delete a receipt file	url
GET	/recurring	List recurring templates	-
GET	/recurring/:id	Get one recurring template	-
POST	/recurring	Create a recurring template	amount, description, type, interval, ...
PATCH	/recurring/:id	Update a recurring template	amount?, isActive?, ...
DELETE	/recurring/:id	Delete a recurring template	-
POST	/cron/recurring	Process due recurring (cron)	X-Cron-Secret header
GET	/health	Health check (no auth)	-

*Summary: 53
endpoints total — 7
public, 30 require JWT
auth, 1 cron job with
special auth, 6 rate-
limited, 4 paginated.*

Generated from the Coin
Toss Finance Manager
codebase.